Introducing Logic Transformer

King-Yin Yan[0009−0007−8238−2442]

general.intelligence@gmail.com

Abstract. Traditional Transformer-based models excel at pattern matching but struggle with compositional generalization and logical coherence, often leading to hallucinations in complex reasoning tasks. In this paper, we introduce the **Logic Transformer**, a novel differentiable neural architecture designed to explicitly perform classical symbolic logic inference (e.g., rule matching and variable substitution). Unlike traditional distributed representations, the Logic Transformer implements a continuous relaxation of symbolic operations. A key innovation is the *cylindrification factor*, which allows logic constants to transition continuously into variables, enabling differentiable unification. Furthermore, we draw an algebraic parallel between our differentiable variable slots and the Self-Attention mechanism, providing a new categorical perspective on neural-symbolic integration. Preliminary experiments on logical reasoning tasks (e.g., bAbI) and language modeling (TinyStories) demonstrate that the Logic Transformer achieves competitive performance while requiring an order of magnitude fewer parameters than traditional Transformers. This high parameter efficiency, coupled with its explicit rule structure, offers a promising direction for building interpretable, resource-efficient, and logically coherent AI systems.

Keywords: Neurosymbolic AI · Differentiable Logic · Parameter Efficiency · Compositional Generalization · Logic Transformer

1 Introduction

The success of Large Language Models (LLMs) based on the Transformer architecture [14] has been unprecedented. However, their reliance on distributed, statistical representations inherently limits their ability to perform rigorous, multi-step logical reasoning. This often manifests as "hallucinations" and a failure in compositional generalization—the ability to combine known concepts into novel configurations [10].

To bridge the gap between the pattern-matching prowess of neural networks and the rigorous coherence of symbolic AI, we propose the **Logic Transformer**. Rather than treating logic as an emergent property of massive scale, we explicitly embed classical symbolic operations—such as rule matching and variable substitution—into the neural architecture via continuous relaxation.

The core contributions of this paper are:

- We introduce a differentiable mechanism for logic rule matching, centered around a novel *cylindrification factor* that smoothly transitions between constant matching and variable substitution.
- We formulate differentiable variable slots using Softmax-based attention, drawing a formal algebraic parallel to the Self-Attention mechanism via string diagrams.
- We demonstrate empirically that this explicit logical structure allows the Logic Transformer to achieve competitive performance on reasoning benchmarks (e.g., bAbI, TinyStories) with an order of magnitude fewer parameters than traditional Transformers.

2 Logic Transformer

In classical logic-based AI, a single step of inference is a mapping $\Gamma \overset{\text{KB}}{\longmapsto} \Gamma + \delta$ where KB is a set of logic **rules** and Γ is a set of ground propositions or **facts**. Γ also called the state $\boldsymbol{x}$ or Working Memory. "Set" implies algebraic structure, ie, permutation invariance of elements. We recognized early on that this has a connection with the **equivariant** structure of Self-Attention; but it was not obvious how to extract further algebraic structures from logic and to apply such symmetries; instead, we found a way to actually implement the syntactic operations of substitutions and logic-rule matching, so the resulting neural network obviously inherits logic structure implicitly. A logic rule is a formula with variables such as:

$$\forall X\ \forall Y\ \forall Z.\quad \text{father}(X, Y) \wedge \text{father}(Y, Z) \Rightarrow \text{grandfather}(X, Z) \tag{1}$$

where the variables must obey strict discipline (X matches to X, etc) as shown by dotted lines.

A key innovation is what I termed the **cylindrification factor** γ, which allows a logic constant to change continuously into a variable, as illustrated in fig. 1.

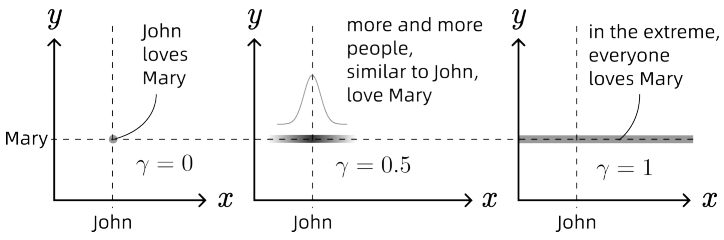
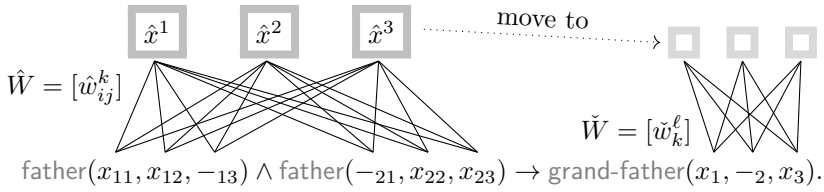


Fig. 1. Illustration of the cylindrification factor γ.

Furthering the idea, the matching (or **unification**) of variables makes use of a fixed number of variable "slots" where we "move" contents in and out of the

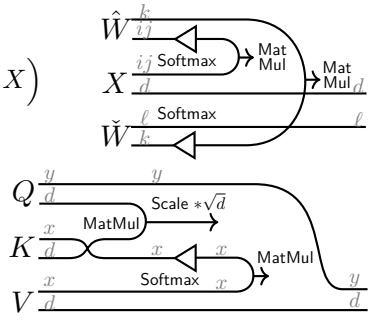
slots by neural weights with **Softmax**, so that the transfer of contents can vary continuously:



(2)

Interestingly, substitution / unification has a form very similar to **Self-Attention** except for the order of softmax and tensor product (with neural string-diagrams explained in the next section):

$$Z := \oslash\check{W} \otimes \left(\oslash\hat{W} \otimes X\right) \qquad \alpha := \left(\oslash \frac{\langle Q, K\rangle}{\sqrt{d_k}}\right) V$$



(3)

The next step is to match Working Memory Elements (WMEs, i.e., propositions) to rule arguments. In classical symbolic AI, this matching process can lead to combinatorial explosion, a challenge historically addressed by algorithms like Rete [4].

Instead of resorting to discrete, computationally prohibitive Cartesian self-products ($X^{\times n}$), the Logic Transformer employs a differentiable approximation. By leveraging the permutation invariance inherent in our architecture, we can efficiently compute the compatibility between WMEs and rule arguments through continuous tensor operations. This allows the network to learn a probabilistic mapping from Working Memory to the most applicable rules, maintaining differentiability while drastically reducing computational overhead.

An initial version of our system can solve TicTacToe quite efficiently. Further testing confirmed that it can solve natural-language tasks such as bAbI [15] and TinyStories [3] with 10x fewer parameters than traditional Transformers (Fig 3L). We attribute this efficiency to the logic structure inherent in the Logic Transformer neural network.

2.1 Source code and experiments

We went through 3 stages of experiments. All source code is under Creative Commons Zero-1.0 license and can be accessed from:
`http://github.com/Cybernetic1/`{insert name}

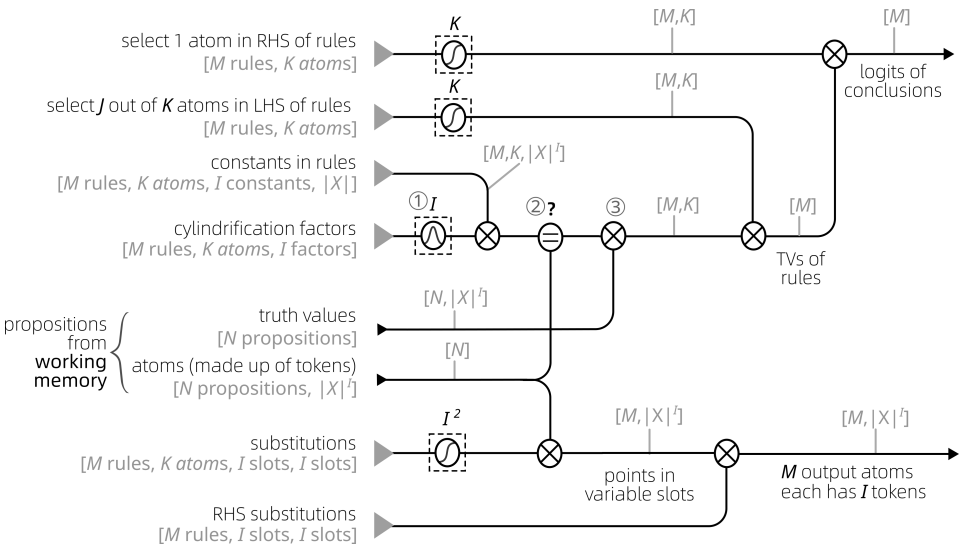


Fig. 2. String-diagram of the Logic Transformer.

1	TicTacToe experiments with traditional neural nets and Transformers	`RL-TTT-experiments`
2	TicTacToe experiments with Logic Transformer	`logic-autoencoder-TTT`
3	AGI experiments with bAbI and Tiny Stories	`AGI-experiments`

Readers can use Copilot to ask questions about our code and even contribute new material. The repository is experimental code and contains >80 Markdown documents, mostly generated by Copilot, which may be difficult for beginners to navigate. Feel free to contact the author for assistance.

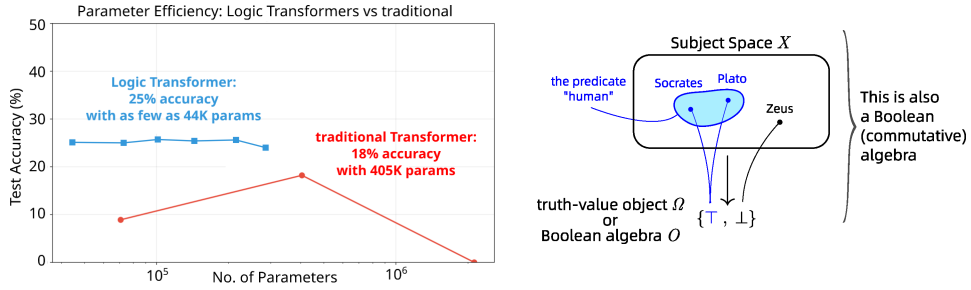


Fig. 3. (Left) Compression ratio, Logic Transformer vs traditional. (Right) Paul Halmos' lifting of propositional logic to predicate logic.

2.2 On the algebraic structure of logic and string diagrams

A natural question arises whether it is possible to simplify the string diagram (Fig. 2) to make it computationally more efficient. Some prior research has been done in this direction. The diagrams in equation (3) are **Functor String Diagrams** as described in [1]. They have the advantage of keeping track of dimensions of tensors as well as preserving equivariant structure – these seem to be all we need for implementing symbolic logic.

Let's digress a little into what we mean by "logic structure." The modern approach is via categorical logic [8] [11], where logic structure is encapsulated as CCCs (**Cartesian closed categories**) or as **topoi** [7]. The categorical approach has Curry-Howard correspondence [12] at its core. A parallel approach is based on relation algebra and leads to the notion of **allegories** [5]; it is inter-translatable with categorical logic. Paul Halmos developed an early idea of algebraic logic based on lifting propositional logic to predicate logic via a morphism that preserves Boolean algebraic structure, as illustrated in Fig. 3R. This idea is later developed into the notion of **hyperdoctrines** by Lawvere.

In the study of neural-symbolic integration, a central concern is the equivariant property $A \wedge B = B \wedge A$, which we believe is also a key feature of Self-Attention [14] and thus Transformers. The categorical way to capture permutation invariance is by **symmetric monoidal categories** (SMCs), which has the visual language of **string diagrams**. **Compact closed** SMCs have dual objects and cup $\cup$ and cap $\cap$ morphisms. These allows to deal with operations such as tensor contraction (eg. dot product) – required by the Attention mechanism. We can **re-write** such string diagrams while preserving their symmetric / algebraic properties, eg [2]. This sounds like a very exciting research direction, though unfortunately because the space of continuous functions has uncountable basis, the completeness of such rewriting cannot be guaranteed. What we have is equational completeness in the sense of $[\![D_1]\!] = [\![D_2]\!] \Rightarrow D_1 \overset{*}{\leftrightarrow} D_2$.

There is a path from SMCs $\rightarrow$ compact closed SMCs $\rightarrow$ Cartesian closed categories $\rightarrow$ topos, with increasingly more structure. We know how to construct neural networks in SMCs so the challenge is how to construct neural networks in toposes. There is some research in this direction [13] [6], which may shed light to our approach.

The string diagram in Fig. 2 is of an older style, as in [9]. The model is impractical as it assumed the naive Cartesian product X^n of Working Memory. We are working with some modified version of it, but the claims of this paper remains basically truthful.

3 Discussion and Future Directions

The explicit logical structure of the Logic Transformer opens several promising avenues for future research:

Hybrid Neurosymbolic Architectures: Because the learned logic rules in our model are cleanly separable entities, the neural network can seamlessly

cooperate with traditional symbolic inference engines (e.g., Prolog or OpenCog). This allows for the rapid, zero-shot incorporation of new symbolic knowledge (e.g., algebraic axioms) directly into the reasoning process, bypassing the need for opaque, long-range dependency learning in traditional LLMs.

Interpretability and Reflection: Unlike the black-box nature of standard Transformers, the activation of specific rule parameters in the Logic Transformer can be directly inspected. This facilitates "reflection," where inference traces can be made into inspectable objects, allowing the system (or human auditors) to verify the logical coherence of its thought processes.

Algebraic Logic and String Diagrams: As discussed in Section **??**, the functorial nature of our architecture suggests a deep connection to Symmetric Monoidal Categories (SMCs). Future work will explore optimizing the string diagram representations to further enhance computational efficiency while preserving equivariant structures.

A Categorical Foundations and String Diagrams: A Beginner's Guide

For readers unfamiliar with category theory, this appendix provides an intuitive introduction to Symmetric Monoidal Categories (SMCs), string diagrams, and the profound categorical insights that underpin the Logic Transformer.

A.1 Intuition: Why Category Theory for Neural Networks?

Modern deep learning frameworks fundamentally operate on directed acyclic graphs of tensor operations. Category theory provides the rigorous mathematical foundation for such graphs. In the context of neural networks:

- **Objects** represent *tensor spaces* (e.g., vector spaces $\mathbb{R}^n$).
- **Morphisms** represent *computations or layers* mapping one tensor space to another.
- **Composition** ($\circ$) represents *sequential execution*.
- **Tensor Product** ($\otimes$) represents *parallel execution*.

A.2 Logical Conjunction: Cartesian vs. Tensor Products

In classical logic, the conjunction ($\wedge$) is naturally modeled by the categorical product (Cartesian product) in a Cartesian Closed Category (CCC). However, in the context of neural networks, representations are often combined via the tensor product ($\otimes$). A key insight of our work is that in certain Topoi or Symmetric Monoidal Categories, the strict Cartesian product can be continuously relaxed into a tensor product. This categorical flexibility is precisely what allows the Logic Transformer to bridge the gap between rigid symbolic logic (where $A \wedge B$ is discrete and strict) and distributed neural representations (where features are entangled via $\otimes$). The *cylindrification factor* γ introduced in the main text can be viewed as the algebraic parameter that navigates this relaxation.

A.3 Categorical Databases, Sheaves, and Logical Instances

A profound connection emerges when we view logical knowledge bases through the lens of categorical databases [13].

- A logical schema (e.g., the ontology of predicates and variables) can be formalized as a category $\mathcal{C}$.
- A specific state of the Working Memory (a set of grounded logical propositions) is then a **set-valued functor** $F : \mathcal{C} \to \mathbf{Set}$, which maps the abstract schema to concrete data instances, much like a database instance populates a database schema.

When we impose logical consistency constraints (e.g., via our differentiable rule matching and sandbox verification), we are effectively defining a Grothendieck topology on $\mathcal{C}$. The valid, logically coherent states of the neural network naturally form a **Sheaf** over this site. This sheaf structure guarantees that local logical consistencies can be uniquely "glued" together into a globally coherent thought process. We believe this topological perspective provides the ultimate mathematical antidote to hallucinations.

A.4 Functor String Diagrams and Tensor Contraction

To visually track the symmetries of variables and the flow of dimensions, we adopt the **Functor String Diagrams** recently formalized by [1]. Unlike traditional string diagrams where wires merely represent abstract objects, functor string diagrams explicitly label wires with their tensor dimensions, allowing us to trace the exact algebraic type of every data stream.

A particularly elegant feature of this formalism is the visualization of **tensor contraction**. In these diagrams, contracting two tensors of matching dimensions is represented by "bending" the wires to connect them, effectively "annihilating" the intermediate dimensions via cup ($\cup$) and cap ($\cap$) morphisms in a compact closed SMC.

- Notably, the **dot product** is simply a special case of tensor contraction.
- The permutation invariance (equivariance) of our Logic Transformer's variable slots is perfectly captured by these diagrams: because the string diagrams are topologically invariant under the continuous deformation of wires (isotopy), the algebraic equivalence of logical permutations is guaranteed visually.
- Crucially, this rigorous dimensional tracking provides a perfect graphical representation of equivariance *without* the need for ad-hoc coloring or manual tracking of variables.

By framing the Logic Transformer in this categorical language, we ensure that the differentiable relaxation of symbolic logic strictly preserves the underlying algebraic symmetries of the knowledge base, paving the way for truly interpretable and mathematically grounded AGI architectures.

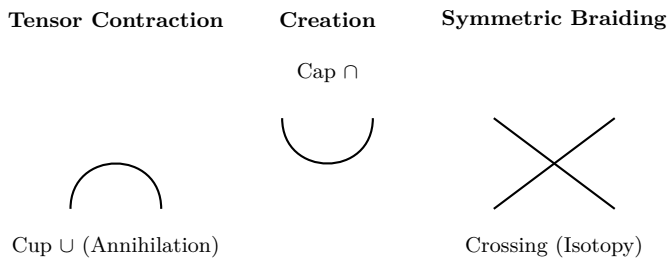


Fig. 4. Topological representation of tensor operations in SMCs. Left & Middle: Tensor contraction (e.g., dot product) via cup and cap morphisms. Right: Symmetric braiding (permutation of variables). The topological invariance of these diagrams under continuous deformation (isotopy) visually guarantees the permutation invariance of logical variables.

A.5 The Visual Language of String Diagrams

String diagrams are a topological visual language for SMCs. They allow us to "see" the algebraic structure of neural architectures without getting bogged down in index notation. The syntax is remarkably simple:

- **Wires** represent data flows (tensors). The thickness or label of a wire indicates the dimension of the tensor.
- **Nodes (or Boxes)** represent operations (morphisms).
- **Plugging wires together** represents sequential composition.
- **Placing diagrams side-by-side** represents the tensor product (parallel computation).
- **Crossing wires** represents the *symmetric braiding*, i.e., permuting the order of tensor dimensions.

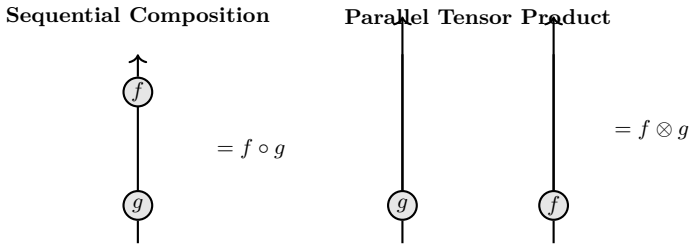


Fig. 5. Basic syntax of string diagrams. Left: Sequential composition. Right: Parallel tensor product.

A.6 Permutation Invariance and Equivariance

A defining feature of the Logic Transformer (and traditional Self-Attention) is **permutation invariance** with respect to the Working Memory (e.g., the logical conjunction $A \wedge B$ is equivalent to $B \wedge A$).

In the language of SMCs, permutation invariance means that the string diagram representing the computation must be *topologically invariant* under the crossing of wires (symmetric braiding). Traditional Transformers achieve this implicitly through the dot-product and softmax. In the Logic Transformer, we explicitly construct morphisms that respect this equivariant structure.

A.7 Compact Closed SMCs and Tensor Contraction

To implement operations like the dot product (which is central to both Self-Attention and our differentiable variable slots), we require a richer structure called a **Compact Closed SMC**. In a compact closed category, every object A has a dual object A^*. This allows for the existence of "cup" ($\cup$) and "cap" ($\cap$) morphisms, which visually represent **tensor contraction** (e.g., summing over a shared index).

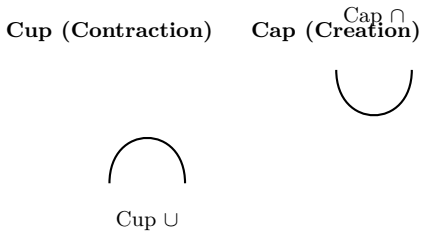


Fig. 6. Topological representation of tensor contraction (dot product) in compact closed SMCs.

The *cylindrification factor* γ and the Softmax-based variable loading mechanism introduced in Section 2 can be rigorously formalized as specific parameterized morphisms within this compact closed SMC. By framing our architecture in this way, we ensure that the differentiable relaxation of symbolic logic strictly preserves the underlying algebraic symmetries of the knowledge base.

References

[1] Vincent Abbott. *Neural Circuit Diagrams: Robust Diagrams for the Communication, Implementation, and Analysis of Deep Learning Architectures*. 2024. arXiv: 2402.05424 [cs.LG]. URL: https://arxiv.org/abs/2402.05424.

[2] Filippo Bonchi et al. “String Diagram Rewrite Theory I: Rewriting with Frobenius Structure”. In: *J. ACM* 69.2 (Mar. 2022). ISSN: 0004-5411. DOI: 10.1145/3502719. URL: https://doi.org/10.1145/3502719.

[3] Ronen Eldan and Mark Li. “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” In: (2023). URL: preprint%20arXiv:2305.07759.

[4] Charles Forgy. “Rete: A Fast Algorithm for the Many Patterns/Many Objects Match Problem”. In: *Artif. Intell.* 19 (1982), pp. 17–37. URL: https://api.semanticscholar.org/CorpusID:62572191.

[5] P.J. Freyd and A. Scedrov. *Categories, Allegories*. North-Holland Mathematical Library. North Holland, 1990. ISBN: 9780080887012. URL: https://books.google.com.hk/books?id=fCSJRegkKdoC.

[6] Bruno Gavranovi et al. *Position: Categorical Deep Learning is an Algebraic Theory of All Architectures*. 2024. arXiv: 2402.15332 [cs.LG]. URL: https://arxiv.org/abs/2402.15332.

[7] Goldblatt. *Topoi – the categorical analysis of logic*. 1984.

[8] Bart Jacobs. *Categorical logic and type theory*. Elsevier, 1999.

[9] Nikhil Khatri et al. “On the Anatomy of Attention”. In: (2024). arXiv: 2407.02423 [cs.LG]. URL: https://arxiv.org/abs/2407.02423.

[10] Brenden Lake and Marco Baroni. “Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks”. In: *International Conference on Machine Learning*. PMLR. 2018, pp. 2873–2882.

[11] Lambek and Scott. *Introduction to higher order categorical logic*. Cambridge, 1986.

[12] Sørensen and Urzyczyn. *Lectures on the Curry-Howard isomorphism*. Elsevier, 2006.

[13] David I. Spivak. *Category Theory for the Sciences*. Cambridge, Massachusetts: MIT Press, 2014.

[14] Vaswani et al. “Attention is all you need”. In: (2017). https://arxiv.org/abs/1706.03762.

[15] Weston, Chopra, and Bordes. “Memory networks”. In: *ICLR (also arXiv)* (2015).